Lab Procedure for Python
Self-Localization

Setup

1. It is recommended that you review Lab 1 – Application Guide before starting this lab.
2. Turn on the QBot Platform by pressing the power button once. To ensure the robot is ready for the lab, check the following conditions.
    a. The LEDs on the robot base should be solid red.
    b. The LCD should display the battery level. It is recommended that the battery level is over 12.5V.
    c. The Logitech F710 joystick's wireless receiver is connected to the QBot Platform. Before use, **always make sure the switch on top is in the X position and that the LED next to the Mode button is off.**
    d. Make sure your computer is connected to the same network that the QBot Platform is on. If using the provided router, the network should be `Quanser_UVS-5G`.
3. Test connectivity to the QBot. Using the IP displayed in the robot's LCD display, enter the following command in your local computer terminal and hit enter: `ping 192.168.2.x`
4. Open localization.py. In Section A change the value of the variable "ipHost" to the IP address of your local Windows machine. To get this IP, open a Command Prompt and enter `ipconfig`. Your IP can be found under IPv4 address and should look like the one the QBot has, `192.168.2.X`.

Introduction to Scan Matching

1. Consider a situation, shown in Figure 1, where the QBot:
    a. Begins at location A
    b. Drives forward
    c. Stops at location B

    Suppose a LiDAR scan is taken at each location, giving you scan A and scan B. Describe how you would relate the data points in scan B to the points in scan A.

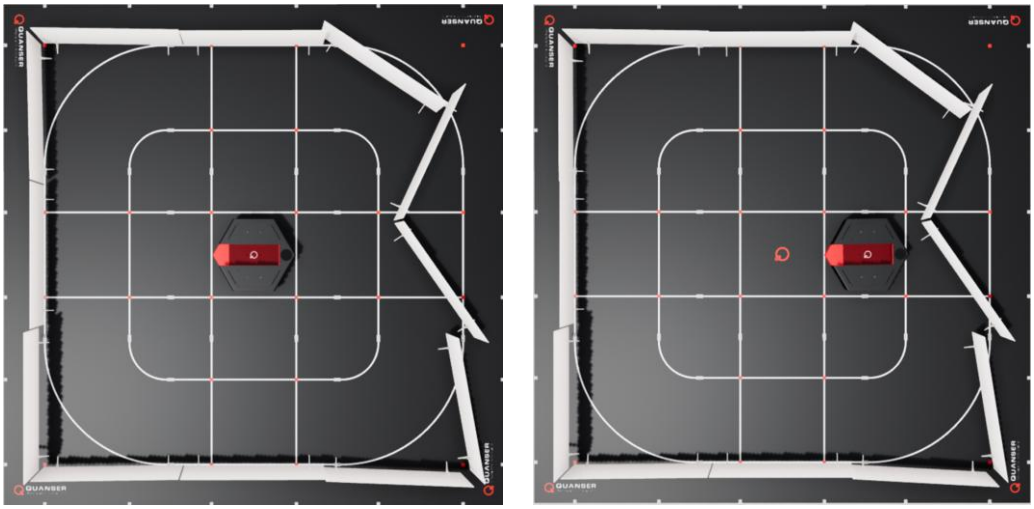


Figure 1. Top view of QBot in location A (left) and location B (right)

2. Consider a situation, shown in Figure 2 below, where the QBot:

    a. Begins facing direction C
    b. Rotates 90 degrees counterclockwise
    c. Stops rotating, so it is now facing direction D

    Suppose a LiDAR scan is taken at each location, giving you scan C and scan D. Describe how you would relate the data points in scan D to the points in scan C. What is the added challenge for rotation compared to translation?

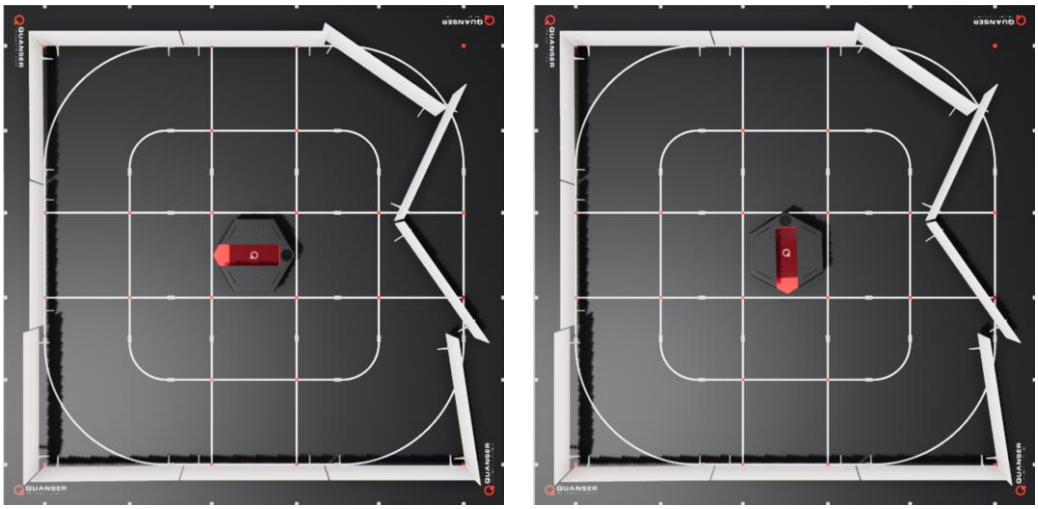


Figure 2. Top view of QBot facing direction C (left) and direction D (right)

Setting up LiDAR Scan Match

3. In this lab you will be exploring LiDAR-based localization using the Scan Match algorithm. Open localization.py. In section B, right click over **`QBPRanging()`** and select "go to definition" to view the class definition within the qbot_platform_functions module. Ensure that the functions **`adjust_and_subsample()`** and **`correct_lidar()`** have been completed according to the instructions in the **Obstacle Detection** lab.

4. In localization.py, fill in Section C to call **`adjust_and_subsample()`** and **`correct_lidar()`** following the **Obstacle Detection** lab if needed.

5. Next, we will be using the Scan Match function implemented in Python to estimate the position of the QBot using LiDAR scan data. Find **`QBPLocalization()`** in section B of localization.py and open the class definition.

6. Scroll down to find the **`scan_match()`** method, which calls the **`match()`** method from the **`Lidar2DMatchScansGrid`** class in Quanser's Image Processing API. The **`match()`** method matches a reference LiDAR scan to the current LiDAR scan, performing a grid matching algorithm to determine the translation and rotation of the current scan relative to the reference. To view descriptions of the method and its inputs, right-click on **`match()`** and click "go to definition".

7. In this lab, we will find the position of the QBot relative to its initial position. To do this, we save a copy of the LiDAR scan in its initial position as the reference scan. Complete the function **`saveRef()`** to save the current scan data. In localization.py, modify the if statement that calls this method in Section D and sets the flag **`refCollected`** once the reference scan has been saved.

8. Return to localization.py. Instead of collecting the reference scan immediately upon entering the main code loop, the reference scan should be collected after a short window of time (a few seconds) for the program to start up. In section D, add code that checks this condition before saving the current LiDAR scan using **`saveRef().`**

9. Return to the code that defines the **`scan_match()`** method. Currently, the parameters passed to the **`match()`** method, **`ref_ranges`**, **`ref_angles`**, **`num_ref_points`**, **`ranges`**, **`angles`**, and **`num_points`**, are set to **`None`**. Modify the code to pass in the correct variables to these parameters.

10. The **`match()`** method starts the grid search at an initial position passed in as the vector **`[x position, y position, theta]`** to the parameter **`initial_pose.`** Complete the function **`scan_match()`** to use the previous estimated position as a starting point for the current search and save it in **`prevPose`**.

11. In Section D of localization.py, modify the line that calls **`scan_match()`** to pass in the current LiDAR scan. The **`transRange`** parameter specifies the extent of the translation search, in m, in the x and y directions. Set this parameter to **`(3.0,3.0)`** in

localization.py. Note as well that the **rotRange** parameter sets the angular search range in radians and is set to a default of 2π.

12. To visualize the QBot's position in the reference scan, each point in the reference scan is converted to Cartesian coordinates and stored in **refX** and **refY**, which are then plotted along with the estimated position on the same XY scope. This code is already implemented within localization.py and qbot_platform_functions.py, do not modify it.

13. Save your changes in both files and run the code by following the steps in the Running Code section. This should open four scope windows on your local Windows machine. The scan match estimates are displayed in the Estimated Pose vs Time and Estimated Pose XY scopes. The Score scope displays a non-normalized confidence value for the match. The Loop Execution Time scope displays the duration of the loop that runs each time data is read from the QBot. Arrange your windows so you can view these scopes at the same time.

14. Drive the QBot around using the joystick controller by holding the left button (LB) to arm the QBot and then moving the left and right joysticks to control movement. Observe the scopes while you drive the QBot around. Which directions correspond to the x and y axes?

15. Observe the Loop Execution Time scope as you drive the QBot around, noting the amount of variation in the execution time. Referring to the code, provide reasons for the variation that we see in the loop timing. Press the right button (RB) to stop the code.

Exploring Parameters of LiDAR Scan Match

16. In this section, we will examine how scan match parameters impact performance. We will use the score, control loop execution time, and estimated location to compare performance. The **resolution** parameter sets the number of grid cells per meter. The **QBPLocalization()** object is initialized with resolution set to 10. Change the resolution to 20 and run the code. Drive the QBot around while observing the scopes. How does changing the resolution impact the performance? If needed, increase the resolution by increments of 5 until you observe a significant effect. Stop your code.

17. With the resolution set to 10, modify the line of code that calls **scan_match** so that **transRange** is set to **(1.0,1.0)**. Run the code and drive the robot around while observing the XY Estimate. How is the performance affected by changing the translation search range?

18. Stop the code by pressing the right button (RB). Ensure that you save a copy of your completed files for review later.

19. Turn OFF the robot by single pressing the power button (do not keep it pressed until it turns off). Post shutdown, all the LEDs should be completely OFF.

Running Code

Note: You will be coding on your local Windows machine and then transferring the code to the QBot Platform. You must go through the steps in this section every time you make changes to the code and want to run it.

1. Open WinSCP, enter the IP address of the QBot Platform for Hostname, and enter "nvidia" for both Username and Password, then click the login button. This opens a connection to the QBot for file transfer.

2. In the WinSCP window, transfer required files to run the application:

    a. Navigate to the directory shown in Figure 3 below, then copy the updated localization.py. In addition, copy the qbot_platform_driver_physical.rt-linux_qbot_platform over. (You can just drag the files over).

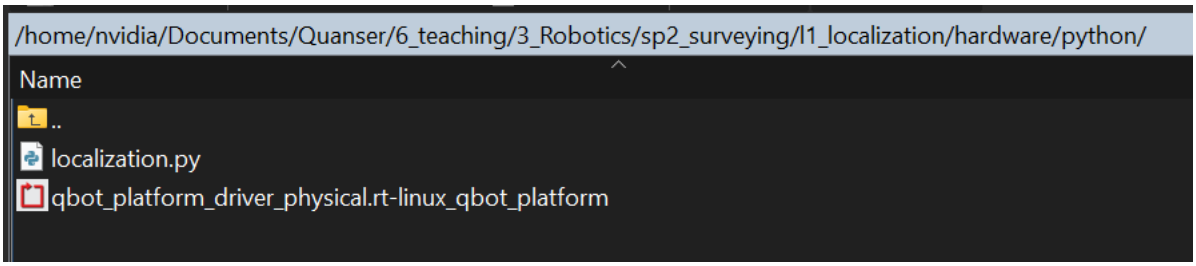


Figure 3. Correct directory for python script

    b. Navigate to the directory shown in Figure 4, or create these directories if they don't exist. Copy over the "hal" and "pal" folder from your local Windows machine, replacing existing files on QBot.

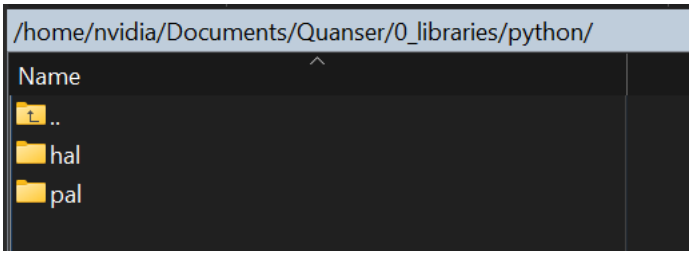


Figure 4. 'hal' and 'pal' folder in correct directory

3. Run observer.py on your local Windows machine first to initiate receiving data feeds from the QBot Platform.

4. Run localization.py on QBot Platform:

    a. Open a PuTTY session by clicking  in the top bars of the WinSCP window.

b. In the PuTTY terminal, enter the password “nvidia”.

c. Navigate to the python script directory from step 2a by using the `cd` command.

d. Run the script using the following command:

```
sudo PYTHONPATH=$PYTHONPATH python3 localization.py
```

e. When the script is run successfully, User LEDs will turn blue.